



SENIOR SOFTWARE ENGINEER INTERVIEW QUESTIONS: DEEP DIVES

Idempotency That Holds

The senior engineer answer sheet

Two identical requests, four milliseconds apart. Your check found nothing, twice.

Almost everyone can describe idempotency. Far fewer can implement one that survives two duplicates arriving at the same instant, which is the only case that matters.

The gap between those two is a single design decision: whether the duplicate is DETECTED by your code or ARBITRATED by the database.

This sheet is the written form of the episode. Database behaviour is sourced to PostgreSQL 17 documentation. Reasoning that is reasoning rather than documented behaviour says so.

In scope

- What makes a key an identity rather than a value
- Why check-then-act fails under concurrency
- The atomic claim, and who wins
- Conflict-aware insert, and what it does not fix
- Putting the claim and the effect in one transaction
- Returning the original result to the loser
- Where the boundary stops: remote calls

Out of scope

- Kafka producer idempotence, which is a different mechanism with the same word
- Distributed locks, which move the problem rather than solving it
- Retry policy and dead letter topics, which are episode 6
- The transactional outbox, which is episode 7

The word idempotent is used for at least three different things in this stack. Saying which one you mean is the first mark of a senior answer.

01 Worked assumptions

Concurrent duplicate requests	32, as in the repo test
Gap between the first two	4 ms
Time for a SELECT to return	about 1 ms
Window a check-then-act leaves open	the whole gap between the read and the write
Duplicates that must be admitted	0
Rows the unique constraint allows	1, decided by the database

These are interview assumptions chosen to make the behaviour concrete. They are not published figures from any provider and no claim is made about any particular deployment.

03 The key is an identity, not a value

A request id generated fresh on every retry identifies the ATTEMPT. It has to identify the INTENT, and survive across retries, restarts and reassignments.

The client owns it, or it is derived deterministically from the payload. Generated server side per call, it identifies nothing.

Beside it, store a fingerprint of the request. Then the same key arriving with different content is a bug you can detect rather than a silent overwrite.

An identifier generated at the point of retry cannot deduplicate retries, because each retry produces a new one.

First principles, not vendor documentation

Storing a fingerprint of the request alongside the key turns key reuse with different content into a detectable conflict rather than an ambiguous one.

First principles, not vendor documentation

04 Why the obvious implementation fails

Read the table, see nothing, do the work, write the row. It works in every test you will write by hand, because those tests are sequential.

Two duplicates arriving concurrently both read, both see nothing, and both proceed. The window is the whole gap between the read and the write, and it is not closable by making the gap smaller.

THE LINE TO REMEMBER. A read followed by a later write can DETECT duplicates in a quiet system. It cannot ARBITRATE two duplicates arriving concurrently.

A read followed by a later write cannot arbitrate concurrent duplicates, because both readers observe the same pre-write state.

First principles, not vendor documentation

The repo test for this episode races thirty two threads on one payment id, and a check-then-act implementation admits more than one of them.

Operational practice, not vendor documentation

05 The atomic claim

Do not ask whether the key exists. Try to INSERT it, and let the unique constraint decide who wins.

One insert succeeds. Every other one violates the constraint. There is no window because there is no read, and the arbitration happens inside the database rather than inside your process.

The winner does the work. The losers do not. This is the whole mechanism, and it is four lines of SQL.

A unique constraint ensures that the values in a column, or a group of columns, are unique among all the rows in the table.

PostgreSQL 17, Constraints

A unique constraint violation is how the database arbitrates a race, and the losing transaction learns it lost without ever having read the winner row.

PostgreSQL 17, Constraints

06 Conflict-aware insert, and the hedge that matters

INSERT ... ON CONFLICT DO NOTHING lets you attempt the claim without treating the loss as an exception, and tells you which happened by whether a row was affected.

It makes the pattern PRACTICAL. It does not make it complete.

That hedge is deliberate. ON CONFLICT resolves the write race. It does not by itself make the surrounding business effect atomic with the claim, and that is the next section.

INSERT ... ON CONFLICT DO NOTHING avoids inserting a row as its alternative action when a uniqueness violation would occur, without raising an error.

PostgreSQL 17, INSERT ... ON CONFLICT

Conflict-aware insert resolves the race on the claim row only; the atomicity of anything else in the same request is a separate question.

First principles, not vendor documentation

07 The claim and the effect are one transaction, or they are nothing

Claim the key, then move the money, then commit. One transaction.

If the effect fails, the transaction rolls back, and the claim rolls back with it. The key is free again and the retry is genuinely a first attempt.

Claim in one transaction and effect in another, and you have built a machine that permanently blocks a payment the moment the second one fails. That failure mode is worse than the duplicate you were preventing.

A transaction bundles multiple steps into a single all-or-nothing operation, so intermediate states are not visible to other transactions and a failure at any step leaves none of them applied.

PostgreSQL 17, Transactions

Committing the claim separately from the effect converts a duplicate risk into a permanent block risk, which is the more expensive of the two for a payment.

First principles, not vendor documentation

08 What the loser returns

The duplicate must not fail. It must return the ORIGINAL result, which means the result has to have been stored with the claim.

A duplicate that returns an error teaches the client to retry, which produces another duplicate. You have built a loop and called it protection.

So the claim row is not a boolean. It holds the key, the fingerprint, the outcome, and when it happened.

Returning an error to a duplicate invites a further retry, so a correct idempotent endpoint returns the stored original outcome instead.

First principles, not vendor documentation

Storing the outcome with the claim in the same transaction is what makes returning it to the loser possible at all.

PostgreSQL 17, Transactions

09 Where the boundary stops

Your local transaction cannot normally make a remote change atomic with it. The word normally is doing real work in that sentence: it is true for an ordinary HTTP call, and there exist protocols where it is less true.

So pass YOUR key to the provider and require them to deduplicate on it. Then the ambiguous case, request sent and response lost, becomes safe to retry.

And be clear about what idempotency does not give you. It does not give you ordering. It does not give you isolation between different keys. It prevents one thing happening twice, and nothing else.

A local database transaction cannot normally make an external HTTP side effect atomic with it, because the remote system is not a participant in that transaction.

First principles, not vendor documentation

Idempotency does not provide ordering or isolation; those are separate properties requiring separate mechanisms.

First principles, not vendor documentation

10 The interview answer

Idempotency means the same logical operation applied more than once has the same effect as applying it once.

It needs a key that identifies the intent, not the attempt, and it needs the duplicate to be arbitrated by a constraint rather than detected by a read.

The claim and the business effect go in one transaction, and the loser gets the original result back rather than an error.

Then name the boundary: remote calls need the provider to deduplicate on your key, because your transaction does not reach them.

Leading with the key and the arbitration, rather than with a definition of the word, is what separates an implemented answer from a memorised one.

First principles, not vendor documentation

Questions to ask before you answer

1. What identifies one logical operation, and who generates it?
2. Does the key survive a retry, a restart and a reassignment?
3. Is the duplicate detected by a read, or arbitrated by a constraint?
4. Are the claim and the effect in the same transaction?
5. What does the second caller get back?
6. Is the payload fingerprinted, so key reuse is visible?
7. Which side effects are outside the transaction boundary?
8. How long do claims live, and what cleans them up?

Everything else

Site

<https://code-with-sam-dev.github.io>

YouTube

<https://www.youtube.com/@CodewithSam-Dev>

LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>

X

<https://x.com/CodeWithSamDev>

TikTok

<https://www.tiktok.com/@codewithsamdev>

Instagram

<https://www.instagram.com/codewithsamdev>

GitHub

<https://github.com/code-with-sam-dev>

Runnable code

<https://github.com/code-with-sam-dev/kafka-payments/tree/main/episode-05-idempotency>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which system you want taken apart next.

Apache Kafka and the Kafka logo are trademarks of the Apache Software Foundation. PostgreSQL and its logo are trademarks of the PostgreSQL Community Association. They appear here only to identify the technology being discussed. Nothing here implies endorsement by, or affiliation with, any of them.

Technical claims verified 2026-09-13 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.